HOSTEL AND MESS MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

MOHANAPRIYA K

(Reg.No:24MCR068)

in partial fulfilment of the requirements

for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTER APPLICATIONS

KONGU ENGINEERING COLLEGE

TRANSFORM YOURSELF

(Autonomous)

PERUNDURAI ERODE – 638 060

MARCH 2026

DEPARTMENT OF COMPUTER APPLICATIONS

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638060

MARCH 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "**HOSTEL AND MESS MANAGEMENT SYSTEM**" is the bonafide record of project work done by the student **MOHANAPRIYA K (24MCR068)** in partial fulfilment for the requirements award of the Degree of Master of Computer Applications, Anna university Chennai during the year 2025-2026.

SUPERVISOR **HEAD OF THE DEPARTMENT**

Date: **(Signature with seal)**

Submitted for the end semester viva voce examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

I affirm that the project report entitled **"HOSTEL AND MESS MANAGEMENT SYSTEM"** being submitted in partial fulfilment of the requirements for the award of Master of Computer Applications is the original work carried out by me. It has not formed the part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Date: **MOHANAPRIYA K**

(Reg.No:24MCR068)

I certify that the declaration made by the above candidates is true to the best of my knowledge.

Date: Name & Signature of the supervisor

[Dr.M.SINDHU]

ABSTRACT

The Hostel & Mess Management System is a Salesforce-based application developed to support the digital management of student accommodations, mess operations, and administrative tasks for educational institutions. The main objective of this system is to provide an efficient and user-friendly platform where students can easily access hostel information, submit complaints, and view mess menus online, while administrators can manage student records, room allocations, fee payments, and complaint resolution through a centralized dashboard.

The system is developed entirely on the Salesforce platform using custom objects, fields, relationships, and automation tools. The frontend is built using Lightning Web Components (LWC) to provide a responsive and interactive user interface, while the backend is developed using Apex programming language to handle server-side logic and business processes. The Salesforce Object Database is used to store student information, room details, mess menus, complaints, and fee payment records. Additional Salesforce features such as validation rules, formula fields, flow automation, and reporting tools are integrated to enhance functionality and usability.

The application consists of multiple integrated modules: Student Management for maintaining student profiles, Room Management for tracking room availability and allocations, Mess Management for daily menu updates, Complaint Management for handling student issues, and Fee Payment Management for tracking financial transactions. The system provides role-based access with different interfaces for administrators, mess staff, maintenance staff, accountants, and students. Digitizing traditional hostel and mess operations, the system reduces manual work, improves data accuracy, and enhances student support services.

The cloud-based platform ensures easy accessibility from anywhere and provides a streamlined solution for managing hostel accommodations, mess services, and administrative tasks efficiently. This project demonstrates how Salesforce's low-code platform can be effectively used to develop practical management systems that improve operational efficiency and user experience.

ACKNOWLEDGEMENT

I express our sincere thanks and heartfelt gratitude to our beloved Correspondent **Thiru.E.R.K.KRISHNAN, M. Com** and other philanthropic trust members of Kongu Vellalar Institute of Technology Trust for having provided with necessary resources to complete this project. I am always grateful to our beloved visionary Principal **Dr.R.PARAMESHWARAN M.E, Ph.D,** and thank him for his motivation and moral support.

I express my deep sense of gratitude and profound thanks to our respected and esteemed Head and Senior Professor **Dr. A. TAMILARASI M.Sc, M.Tech, M.Phil, Ph.D,** Department of Computer Applications for her valuable commitment and guidance for this project.

I would also like to express my heartfelt gratitude and sincere thanks to our dedicated project coordinators **Mrs.S.HEMALATHA MCA,** Assistant Professor (SLG) and **Dr. M. SINDHU MCA, Ph.D,** Assistant Professor Department of Computer Applications, Kongu Engineering College who have motivated us in all aspects for completing the project in scheduled time.

I would like to express my gratitude and sincere thanks to my project guide, **Dr. M. SINDHU MCA, Ph.D,** Assistant Professor Department of Computer Applications, Kongu Engineering College for giving her valuable guidance and suggestions which helped me in the successful completion of the project.

I owe my deepest gratitude to my parents for empowering me to overcome every challenge in all my endeavors. I bow my head with sincere thanks to all those who extended their warm support and guidance, helping us accomplish and complete our work successfully.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	iv
	ACKNOWLEDGEMENT	v
	LIST OF FIGURES	viii
	LIST OF TABLES	ix
	LIST OF ABBREVIATIONS	x
1	**INTRODUCTION**	1
	1.1 OVERVIEW	1
	1.2 EXISTING SYSTEM	2
	1.3 DRAWBACKS OF EXISTING SYSTEM	2
	1.4 PROPOSED SYSTEM	3
	1.5 ADVANTAGES OF PROPOSED SYSTEM	3
2	**SYSTEM ANALYSIS**	5
	2.1 IDENTIFICATION OF NEED	5
	2.2 FEASIBILITY STUDY	5
	2.2.1 Economic Feasibility	6
	2.2.2 Operational Feasibility	6
	2.2.3 Technical Feasibility	7
	2.3 SOFTWARE REQUIREMENT SPECIFICATION	8
	2.3.1 Software Requirements	8
	2.3.2 Hardware Requirements	9
	2.4 SOFTWARE DESCRIPTION	8
	2.4.1 Front End	9
	2.4.2 Back End	9
	2.4.3 Database	10
3	**SYSTEM DESIGN**	11
	3.1 MODULE DESCRIPTION	11
	3.1.1 Student Management Module	11

	3.1.2 Room Management Module	11
	3.1.3 Mess Management Module	12
	3.1.4 Complaint Management Module	12
	3.1.5 Fee Management Module	12
	3.1.6 Notification Module	13
	3.2 DATA FLOW DIAGRAM	13
	3.2.1 DFD Level 0	13
	3.2.2 DFD Level 1	14
	3.3 SYSTEM FLOW DIAGRAM	15
	3.4 INPUT DESIGN	16
	3.5 OUTPUT DESIGN	18
	3.6 ER DIAGRAM	20
	3.7 DATABASE DESIGN	20
	3.8 TABLE DESIGN	21
4	**IMPLEMENTATION**	24
	4.1 CODE DESCRIPTION	24
	4.2 STANDARDIZATION OF CODING	24
	4.3 ERROR HANDLING	25
5	**TESTING AND RESULTS**	27
	5.1 TESTING	27
	5.1.1 Unit Testing	27
	5.1.2 Integration Testing	28
	5.1.3 Validation Testing	29
6	**CONCLUSION AND FUTURE ENHANCEMENTS**	31
	6.1 CONCLUSION	31
	6.2 FUTURE ENHANCEMENTS	31
	APPENDICES	
	A. SAMPLE CODING	
	B. SCREENSHOTS	
	REFERENCES	46

LIST OF FIGURES

FIGURE NO	NAME OF FIGURE	PAGE NO
3.1	Data Flow Diagram Level 0	13
3.2	Data Flow Diagram Level 1	14
3.3	System Flow Diagram	15
3.4	Student Registration Form	16
3.5	Room Management Page	17
3.6	Warden Login Page	17
3.7	Admin Home Page	18
3.8	Student Detail Page	19
3.9	Complaint Registered Page	19
3.10	Entity Relationship Diagram	20
B.1	Hostel and Mess Portal Home Page	41
B.2	Student Record Page	42
B.3	Student Detail Page	42
B.4	Student Related Page	43
B.5	View Notification	43
B.6	Complaint Register Page	44
B.7	Login Page	44
B.8	Profile Page	45

LIST OF TABLES

TABLE NO	NAME OF TABLE	PAGE NO
3.1	Student Object (Student__c)	21
3.2	Room Details Object (Room_Details__c)	21
3.3	Mess Details Object (Mess_Details__c)	22
3.4	Complaint Details Object (complaint_Details__c)	22
3.5	Fee Payment Object (Fee_payment__c)	23
5.1	Unit Testing	28
5.2	Integration Testing	29
5.3	Validation Testing	30

LIST OF ABBREVATIONS

ABBREVIATIONS	EXPANSIONS
LWC	Lightning Web Components
API	Application Programming Interface
UI	User Interface
DB	Database
SDG	Sustainable Development Goals
CRM	Customer Relationship Management
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
JS	JavaScript

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Hostel and mess management is an important part of educational institutions that provide residential facilities for students. Proper management of student accommodation, room allocation, mess services, fee collection, and complaint handling is necessary to maintain a comfortable living environment for students and ensure smooth administrative operations. In many institutions, hostel and mess management is still handled using traditional manual methods such as paper registers, spreadsheets, and physical files. Students often need to visit the administrative office to apply for hostel accommodation, report complaints, or check mess fees. At the same time, administrators face difficulties in tracking room availability, managing mess schedules, recording fee payments, and maintaining complaint records.

Manual processes often create several problems such as difficulty in tracking occupied or available rooms, delays in resolving student complaints, errors in payment records, and lack of centralized student information. With the advancement of cloud-based platforms like Salesforce, educational institutions can improve hostel management through digital systems that provide better accessibility, transparency, and efficiency. Salesforce offers a low-code platform that allows developers to build customized applications with minimal programming knowledge.

The Hostel and Mess Management System is developed as a Salesforce-based application that uses custom objects, fields, relationships, and automation features. The system integrates multiple functions such as student registration, room allocation, mess menu management, fee tracking, and complaint handling into a single platform. This system provides role-based access where administrators can manage student records, rooms, mess information, payments, and complaints, while students can view their room details, check mess menus, submit complaints, and monitor their fee status.

1.2 EXISTING SYSTEM

In many educational institutions, the existing hostel and mess management system still depends on traditional manual processes. Students usually fill out paper application forms to apply for hostel accommodation, and administrators manually assign rooms based on availability. This process often takes more time and may lead to errors because there is no real-time system to check which rooms are occupied or available. If any changes are made to the menu, the notice boards must be updated manually. In addition, the calculation of monthly mess fees is often done using spreadsheets or manual records, which increases the chances of calculation errors.

Complaint handling is another area where manual systems create problems. Students typically report issues such as room maintenance, cleanliness, or food quality by writing their complaints in registers or sending emails to administrators. However, there is no proper system to track the progress or status of these complaints, which may lead to delays in resolving problems and dissatisfaction among students. Hostel and mess fees are usually collected through cash or bank transfers and recorded in registers or spreadsheets.

In addition, generating reports related to hostel occupancy, fee collections, pending complaints, or mess expenses becomes difficult because data must be collected from different manual records. This process is time-consuming and increases the possibility of errors. Therefore, the existing system is not efficient for managing hostel and mess operations in a structured and organized way.

1.3 DRAWBACKS OF EXISTING SYSTEM

The traditional hostel management system suffers from several limitations:

- Manual record keeping leads to data entry errors and lost information
- No real-time visibility into room availability and occupancy status
- Delayed complaint resolution due to lack of systematic tracking
- Difficult fee tracking with no automated reminders for pending payments
- Time-consuming report generation requiring manual data compilation
- Lack of centralized student information across different hostel functions
- No online access for students to view their details, complaints, or fee status
- Difficulty in historical data retrieval for audits or analysis

1.4 PROPOSED SYSTEM

The proposed Hostel and Mess Management System is developed using the Salesforce platform to digitize and simplify hostel operations. It leverages powerful Salesforce features such as custom objects, lookup relationships, validation rules, formula fields, flow automation, and Lightning App Builder to manage student records, room allocation, mess services, complaints, and fee payments in a centralized, cloud-based system. The platform's low-code development approach enables rapid implementation while ensuring scalability, security, and reliability, allowing educational institutions to transform their traditional manual processes into an efficient digital workflow without extensive programming knowledge.

The system helps administrators handle hostel activities more efficiently by automating routine tasks and providing real-time visibility into all operations. The centralized database eliminates data silos and ensures accurate, up-to-date information for all stakeholders. Role-based access control allows administrators, mess staff, maintenance staff, accountants, and students to access only the data relevant to their roles, ensuring security and efficiency. Students benefit from self-service access through their portal to view room details, check mess menus, submit complaints, and track fee payments without visiting administrative offices.

The system maintains a comprehensive student database with personal details, contact information, enrollment data, and unique Student IDs generated automatically. Student status tracking with active/inactive checkboxes automatically updates room availability and fee calculations when students check out.

The system also provides fee payment tracking with automated calculations of paid and pending amounts, real-time financial status updates, and automatic receipt generation. Email notifications keep students informed about complaint resolutions and fee reminders. Salesforce security features protect sensitive data through field-level security, while audit trails ensure accountability. Reporting capabilities allow administrators to monitor fee collections, track complaint resolution times, analyze occupancy trends, and generate custom reports for informed decision-making and continuous service improvement.

1.5 ADVANTAGES OF PROPOSED SYSTEM

- Provides a centralized platform for managing all hostel and mess operations
- Real-time visibility into room availability and occupancy status
- Automated complaint tracking with status updates and notifications
- Easy mess menu management with historical tracking
- Automated fee tracking with pending payment identification
- Quick report generation through Salesforce reporting tools
- Improved data accuracy through validation rules
- Enhanced communication between students and administration
- Mobile accessibility through Salesforce mobile app
- Scalable architecture that can accommodate future enhancements
- Reduced manual workload for administrative staff
- Better student experience through timely complaint resolution
- Audit trails for all data changes

  Cost-effective using Salesforce's low-code platform

SUMMARY

This chapter discussed the need to modernize hostel and mess management using Salesforce technology. It explained the limitations of traditional manual systems, such as inefficient record keeping, delayed complaint handling, and lack of centralized data management. These issues highlight the importance of implementing a digital solution. The proposed Hostel and Mess Management System was introduced as a Salesforce-based platform that helps manage student records, room allocation, mess services, complaints, and fee tracking in an organized way. The next chapter focuses on system analysis, including feasibility study and system requirements.

CHAPTER 2

SYSTEM ANALYSIS

2.1 IDENTIFICATION OF NEED

Educational institutions that provide hostel facilities often face several challenges in managing student accommodation, mess services, and related administrative tasks. As the number of students increases, managing hostel activities manually becomes difficult and time-consuming. Manual systems can lead to errors in maintaining records and make it hard to manage a large amount of student data efficiently. Administrators also require real-time information such as room availability, occupancy status, pending fee payments, and complaint updates in order to make quick and effective decisions. At the same time, students today expect digital services that allow them to easily access information, check hostel details, and submit complaints online without visiting the administrative office.

Institutions are also required to maintain proper records for inspections, audits, and reporting purposes. Manual systems make it difficult to keep accurate and updated records. In addition, manual processes consume a significant amount of staff time that could otherwise be used to improve student services and support. Therefore, there is a strong need for a digital hostel and mess management system that can organize and manage all related information in a centralized platform. Such a system should be easy to use, reliable, cost-effective, and capable of handling a large amount of data while improving the overall efficiency of hostel management.

2.2 FEASIBILITY STUDY

A feasibility study is an important stage in system analysis that determines whether the proposed system can be successfully developed and implemented. It evaluates the practicality, effectiveness, and sustainability of the system before actual development begins. The feasibility study ensures that the project is technically achievable, economically viable, and operationally acceptable. In this project, the feasibility study is conducted to evaluate the Hostel & Mess Management System from three major perspectives: economic

feasibility, operational feasibility, and technical feasibility. Economic feasibility analyzes whether the benefits of the system justify the development and maintenance costs. Operational feasibility focuses on how effectively the system can function within the existing institutional environment and whether users can easily adopt it. Technical feasibility evaluates whether the required platform resources and technical expertise are available for development and maintenance.

Three considerations involved in feasibility are:

- Economic Feasibility
- Operational Feasibility
- Technical Feasibility

2.2.1 Economic Feasibility

Economic feasibility evaluates whether the proposed system is financially practical and cost-effective. It compares the development and operational costs of the system with the expected benefits it provides. The proposed system is economically feasible because it is developed on the Salesforce platform, which many educational institutions may already have licensed. The system leverages existing Salesforce subscriptions, eliminating the need for additional software purchases. The low-code development approach using custom objects, fields, and flows significantly reduces development time and costs compared to traditional coding.

Therefore, the proposed system is economically feasible and beneficial for educational institutions.

2.2.2 Operational Feasibility

Operational feasibility refers to the ability of the proposed system to function effectively within the existing operational environment and be easily accepted by users. The proposed Hostel & Mess Management System is designed with a simple and user-friendly Lightning interface that allows both administrators and students to use the system without difficulty. Administrators can easily manage student records, room allocations, mess menus, complaints, and fee payments through intuitive record pages and dashboards.

The system automates many routine tasks such as room availability calculations, complaint status tracking, and fee pending notifications. This reduces manual workload and improves the overall efficiency of hostel operations. Since the system integrates smoothly with existing institutional processes, it does not require significant changes in daily workflow. Staff with basic computer skills can quickly learn to use the system with minimal training.

Therefore, the system is operationally feasible and can be successfully adopted by hostel administrators and students.

2.2.3 Technical Feasibility

Technical feasibility evaluates whether the required technical resources, tools, and expertise are available for developing and maintaining the system.The proposed system is technically feasible as it is developed on the Salesforce platform, which provides a robust, secure, and scalable infrastructure. Salesforce offers all necessary components including custom objects, fields, relationships, validation rules, flows, and reporting tools. These features are well-documented and supported by a large community of developers and administrators.The system can be accessed through any modern web browser and is also available on mobile devices through the Salesforce mobile app. The development tools required for the project are built directly into the Salesforce platform, eliminating the need for external development environments. Salesforce's reliable cloud infrastructure ensures high availability, automatic backups, and data protection. The platform handles scalability automatically, so the system can easily accommodate growing numbers of students, rooms, and records without performance degradation.

Hence, the proposed system meets all technical requirements and can be successfully implemented and maintained.

2.3 SOFTWARE REQUIREMENT SPECIFICATIONS

Software Requirement Specification (SRS) defines the functional and non-functional requirements of the proposed system. It provides a clear description of the system behavior and identifies the platform resources required for successful implementation. The SRS acts as a blueprint for system design, development, and testing, ensuring that the final application meets user requirements and project objectives. In this project, the SRS describes the

Salesforce platform features, tools, and configurations used to develop the Hostel & Mess Management System. Software Requirement Specification (SRS) defines the functional and non-functional requirements of the proposed system. It provides a clear description of the system behavior and identifies the platform resources required for successful implementation. The SRS acts as a blueprint for system design, development, and testing, ensuring that the final application meets user requirements and project objectives. In this project, the SRS describes the Salesforce platform features, tools, and configurations used to develop the Hostel & Mess Management System.

2.3.1 Software Requirements

The following software requirements are necessary for the development and execution of the proposed system:

- **Operating System:** Windows 10 / Linux / macOS
- **Frontend Technology:** Lightning Web Components (LWC)
- **Backend Technology:** Apex Programming Language
- **Database :** Salesforce Object Database
- **Authentication Mechanism:** Salesforce Built-in Authentication, OAuth 2.0
- **Web Browser:** Google Chrome, Mozilla Firefox, Microsoft Edge
- **Development Tools:** Salesforce Setup Menu, Developer Console, Visual Studio Code with Salesforce Extensions, Salesforce CLI (Command Line Interface)
- **Testing Tool:** Apex Test Execution, Salesforce Inspector

These software tools enable efficient development, testing, deployment, and maintenance of the system.

2.3.2 Hardware Requirements

The following hardware requirements are necessary for the development and execution of the proposed system:

- **Device Type:** Laptop / Desktop Computer
- **Processor:** Intel Core i5
- **Processor Speed:** 2.0 GHz or higher

- **Installed RAM:** 8.00 GB (minimum 4 GB required)
- **System Type:** 64-bit Operating System, x64-based Processor
- **Storage:** Minimum 256 GB Hard Disk / SSD
- **Display Resolution:** 1366 × 768 pixels or higher

These hardware specifications ensure smooth performance, efficient processing, and reliable execution of the proposed system.

2.4 SOFTWARE DESCRIPTION

2.4.1 Front End

The front end of the proposed system is developed using Lightning Web Components (LWC), a modern JavaScript framework provided by Salesforce for building responsive and interactive user interfaces. LWC uses core web standards such as JavaScript, HTML, and CSS while leveraging Salesforce's powerful features, providing excellent performance and seamless integration with the platform.

Lightning Web Components follow a component-based architecture, allowing developers to create reusable UI components that simplify application development and maintenance. Each component encapsulates its own HTML template, JavaScript logic, and CSS styles, making the code modular, organized, and easy to update.

2.4.2 Back End

The back end of the system is developed using Apex, a strongly typed, object-oriented programming language provided by Salesforce for building server-side logic and business processes. With syntax similar to Java and execution entirely within Salesforce’s multitenant infrastructure, Apex enables complex business logic that cannot be achieved through declarative tools alone. Apex triggers execute automatically before or after record events such as insert, update, or delete to validate data, update related records, enforce business rules, and maintain data integrity across objects. Beyond triggers, Apex supports asynchronous processing, web service callouts, and robust developer tooling, allowing for scalable integrations and modular, maintainable code architecture.

A critical aspect of Apex development is adherence to Salesforce's governor limits, which impose constraints on resources such as query counts, heap size, and CPU time to ensure equitable performance in the multitenant environment. Developers must optimize code through techniques like bulkification and selective querying to operate within these limits while maintaining functionality. Additionally, Apex enforces a mandatory testing framework requiring at least 75% code coverage for production deployment, promoting reliability and resilience. Together, these features enable Apex to serve as a powerful, secure foundation for automating sophisticated business processes, enforcing data integrity, and extending the Salesforce platform beyond the capabilities of declarative configuration alone.

2.4.3 Database

The database used in the proposed system is the Salesforce Object Database, a cloud-based, multi-tenant relational database designed specifically for the Salesforce platform. Data is stored in objects (similar to tables) and records (similar to rows), with automatic backups, high availability, and enterprise-grade security.The system uses custom objects including Student_Details__c for student information, Room_Details__c for room details, Mess_Details__c for mess menus, Complaint_Details__c for complaints, and Fee_Payment__c for fee transactions. Objects are connected through lookup relationships, such as linking complaints to students and students to rooms, ensuring data integrity and enabling complex queries.

SUMMARY

This chapter presented the system analysis of the proposed Hostel & Mess Management System. It identified the need for digital transformation in hostel operations due to limitations of manual systems such as difficulty in tracking room availability, delays in complaint resolution, errors in fee records, and lack of centralized information. A feasibility study evaluated the project from economic, operational, and technical perspectives, confirming that the system is practical, cost-effective, and technically achievable on the Salesforce platform. The chapter also described the software and hardware requirements, including Lightning Web Components for frontend, Apex for backend, and the Salesforce Object Database for data storage.

CHAPTER 3

SYSTEM DESIGN

3.1 MODULE DESCRIPTION

The Hostel & Mess Management System is designed using a modular architecture to ensure better organization, maintainability, and scalability of the system. Each module performs a specific function in the hostel management process, allowing the system to efficiently handle student registration, room allocation, mess operations, complaint management, and fee tracking. The modular design also simplifies development, testing, and future enhancements of the system.

3.1.1 Student Management Module

The Student Management Module is responsible for maintaining complete information about all students residing in the hostel. It acts as a centralized database where administrators can store and manage student details such as student name, email address, phone number, course details, join date, and current status. Each student is assigned a unique Student ID that helps in identifying and tracking individual records easily. The module is also connected with the Room Management Module through lookup relationships, which helps in assigning rooms to students and maintaining accurate hostel occupancy records.

3.1.2 Room Management Module

The Room Management Module manages all hostel room information and keeps track of room availability. The system automatically calculates the number of available beds by comparing the room capacity with the number of students assigned to that room. This helps administrators quickly identify which rooms are available and which are fully occupied. The module also works together with the Student Management Module to assign students to rooms efficiently while preventing over-allocation of rooms beyond their capacity.

3.1.3 Mess Management Module

The Mess Management Module handles all activities related to the hostel mess and meal services. This module allows administrators or mess staff to create and update daily meal menus such as breakfast, lunch, and dinner along with the food items served. Students can easily view the menu through the system without depending on physical notice boards. The module also maintains records of mess services and tracks monthly mess fees, which can be linked with the Fee Payment Module. By maintaining menu history, the system helps in planning future meal schedules and managing mess operations more efficiently.

3.1.4 Complaint Management Module

The Complaint Management Module provides a platform for students to submit complaints related to hostel facilities or mess services. Students can report issues such as room maintenance problems, electricity issues, cleanliness concerns, or food quality through an online form. Each complaint is recorded with details such as complaint type, description, and submission date. Administrators can track the complaint status through stages such as New, In Progress, and Resolved. This module helps improve communication between students and hostel staff and ensures that issues are resolved quickly and efficiently.

3.1.5 Fee Management Module

The Fee Payment Module manages and records all hostel and mess fee payments made by students. It stores important payment information such as payment amount, payment date, payment method, and payment status. Each payment record is linked to the respective student, allowing administrators to track the complete financial history of every student. The system can also identify pending or overdue payments and help administrators send reminders when necessary. This module ensures accurate fee tracking and helps in maintaining proper financial records for hostel management.

3.1.6 Notification Module

The Notification Module is responsible for informing students and administrators about important updates in the hostel and mess management process. Notifications may include complaint status updates, fee payment confirmations, room assignment details, and mess menu changes. The notification mechanism helps keep users informed about the status of their requests and improves communication between students and hostel staff.

3.2 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation that shows how data moves within a system. It illustrates the flow of information between external entities, processes, and data storage components. DFDs help in understanding how data enters the system, how it is processed, and how it is stored or delivered to users.By using DFDs, developers and stakeholders can easily visualize the system's operations and identify the relationships between different modules. It simplifies complex system processes and helps in designing an efficient and well-structured application.

3.2.1 DFD Level 0

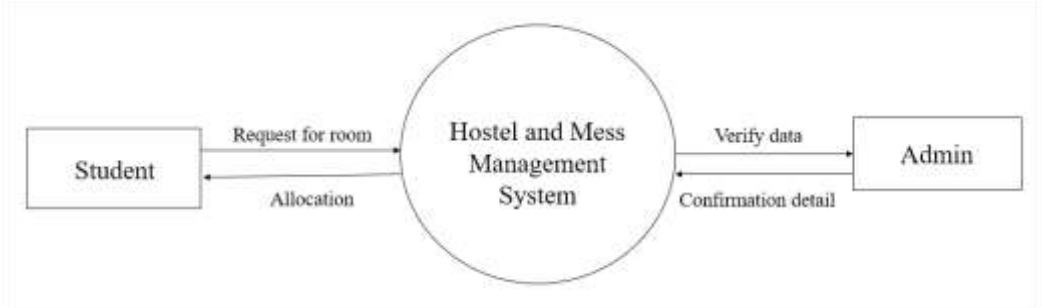

Fig 3.1 Data Flow Diagram Level 0

DFD Level 0, also known as the Context Diagram, provides a high-level overview of the entire system. It represents the whole application as a single process and shows how external entities interact with the system. This level focuses only on major data flows and does not include detailed internal processes.

In the proposed Hostel & Mess Management System, the main external entities interacting with the system are Student, Administrator, Mess Staff, Maintenance Staff, and Accountant. All interactions occur through the central Salesforce-based system.

In Figure 3.1 shows the Level 0 Data Flow Diagram (DFD) of the Hostel & Mess Management System. In this diagram, the main external entities interacting with the system are Student, Admin, Staff, and Accountant.

3.2.3 DFD Level 1

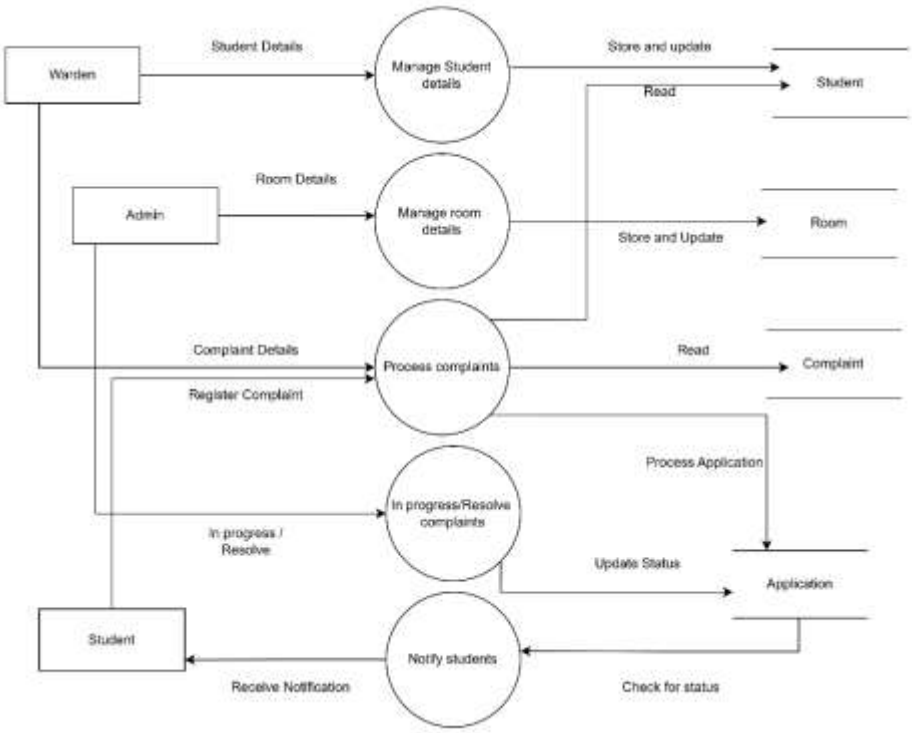

Figure 3.2 Data Flow Diagram Level 0

DFD Level 1 shows the internal working of the Hostel & Mess Management System. It breaks the system into major processes and explains how data moves between users, system modules, and databases. In this system, the main processes are Student Management, Room Management, Mess Management, Complaint Management, Fee Payment Management, and Report Generation.

3.3 SYSTEM FLOW DIAGRAM

A System Flow Diagram visually represents the step-by-step execution of processes within a system. The System Flow Diagram of the Hostel & Mess Management System illustrates how users interact with the system and how operations progress through different modules until the outcome.

The system flow begins when the user logs in with their credentials. Once authenticated, the user is directed to their respective dashboard based on their role. From the dashboard, the admin manages students, rooms, complaints, and generates reports. The student views room details, checks mess menus, submits complaints, and views fee status. Mess staff updates daily mess menus and meal details.

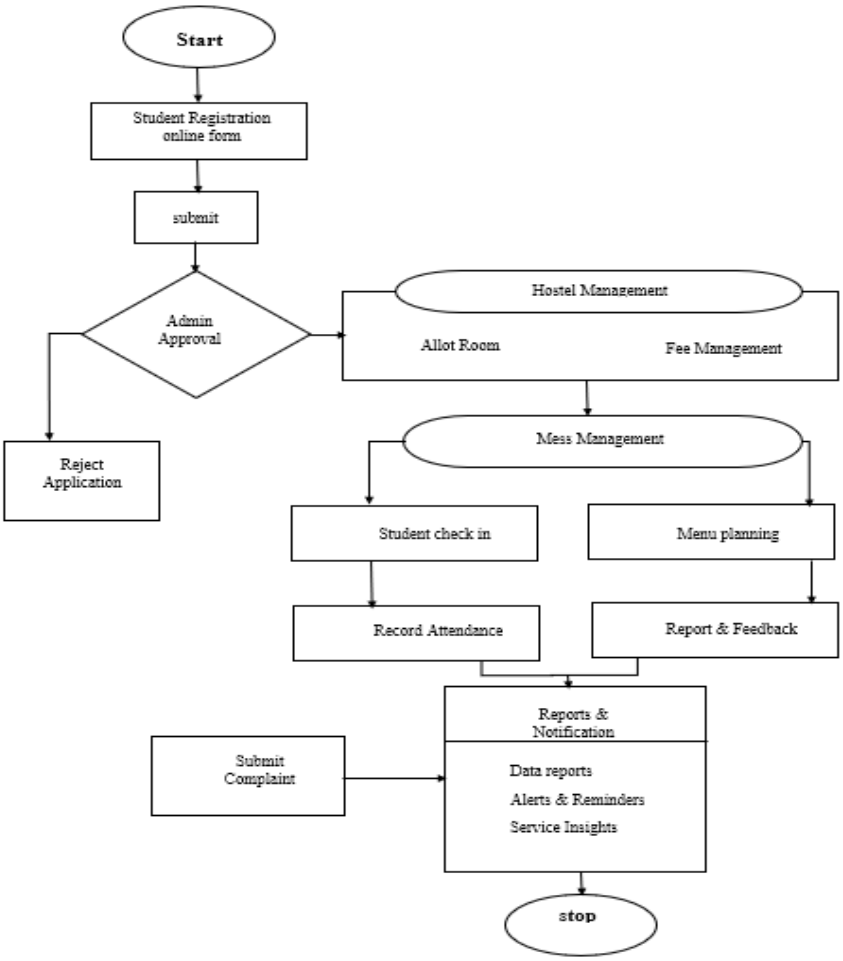

Fig 3.3 System Flow Diagram

The System Flow Diagram of the Hostel and Mess Management System clearly depicts the sequential flow of operations, user interactions, and communication between different system modules. It provides a clear understanding of how the system functions from the login process to the management of hostel activities such as room allocation, mess services, complaint handling, and fee payments.

3.4 INPUT DESIGN

Input design focuses on collecting accurate and valid data from users so that the system can process hostel and mess management operations effectively. The Hostel & Mess Management System uses structured input forms developed using Lightning Web Components to collect information from students, administrators, mess staff, maintenance staff, and accountants. Input validation mechanisms ensure that only valid data is accepted by the system.

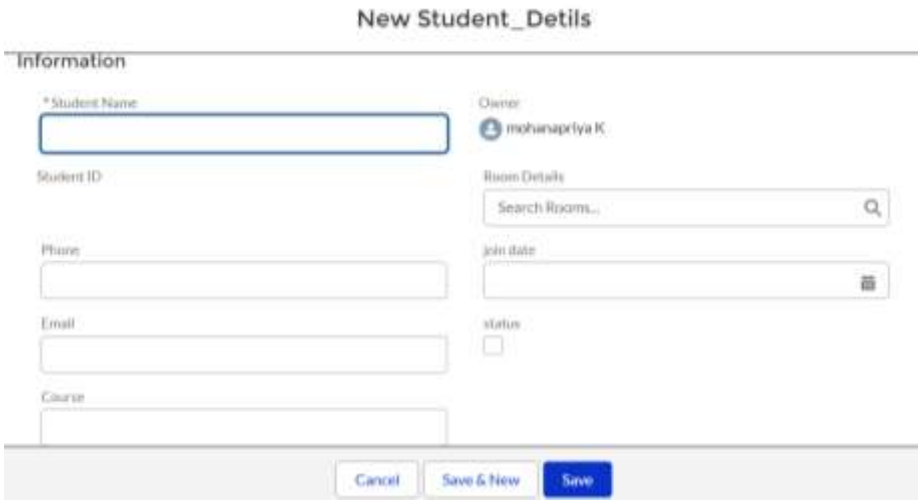

Figure 3.4 Student Register Form

This screen allows students to enter personal, academic, and financial information required for hostel accommodation and services. The form includes fields such as student name, email address, phone number, course of study, join date, and room preferences. Proper validations ensure that the entered information follows the required format, with email format checking, phone number digit validation, and required field enforcement to maintain data accuracy.

.

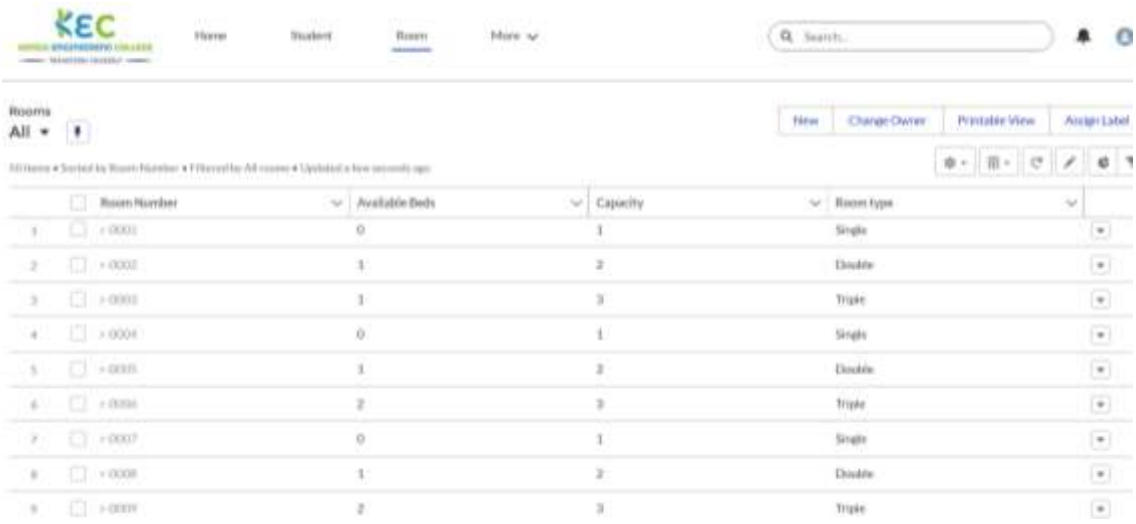

Figure 3.5 Room Management Screen

This screen is used by the admin to create and manage hostel rooms. It includes input fields such as room number, floor location, total capacity, room type (Single, Double, Triple, Dormitory), and current availability status. Validation ensures that room numbers are unique, capacity is a positive number, and all required fields are completed before saving.

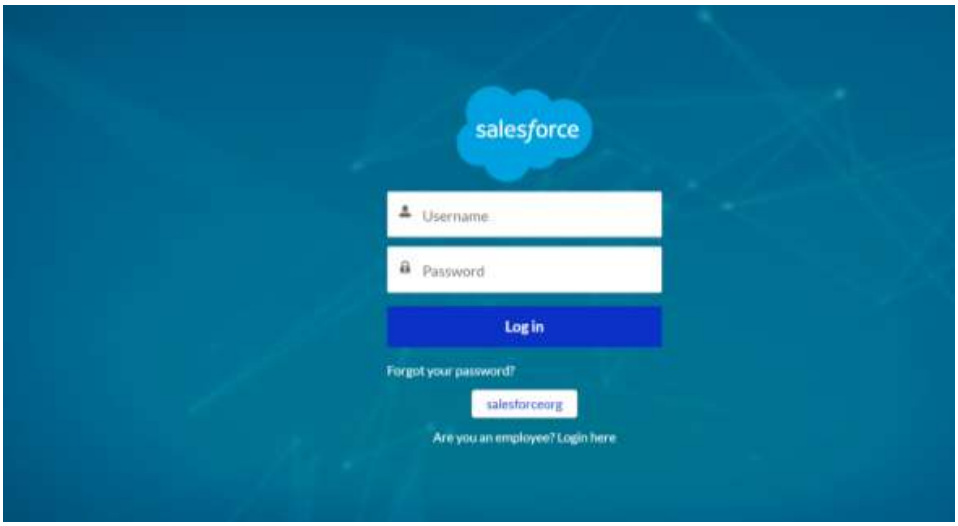

Figure 3.6 Hostel Warden Login Screen

This screen serves as the secure entry point to the system, allowing authorized users to authenticate their identity before gaining access to protected resources and functionality. It presents a clean, intuitive interface that includes essential authentication fields, such as username and password, which are validated against stored credentials to verify user identity. Additional security measures may be integrated into the screen, including multi-factor authentication options, password masking for privacy, account lockout protections after repeated failed attempts, and secure session management to prevent unauthorized access.

3.5 OUTPUT DESIGN

Output design focuses on presenting processed information in a clear and meaningful format for users. The Hostel & Mess Management System generates outputs that help students view room details, check mess menus, track complaint status, and monitor fee payments. Administrative users can view student records, monitor room occupancy, track complaint resolution, and manage fee collections through output screens.

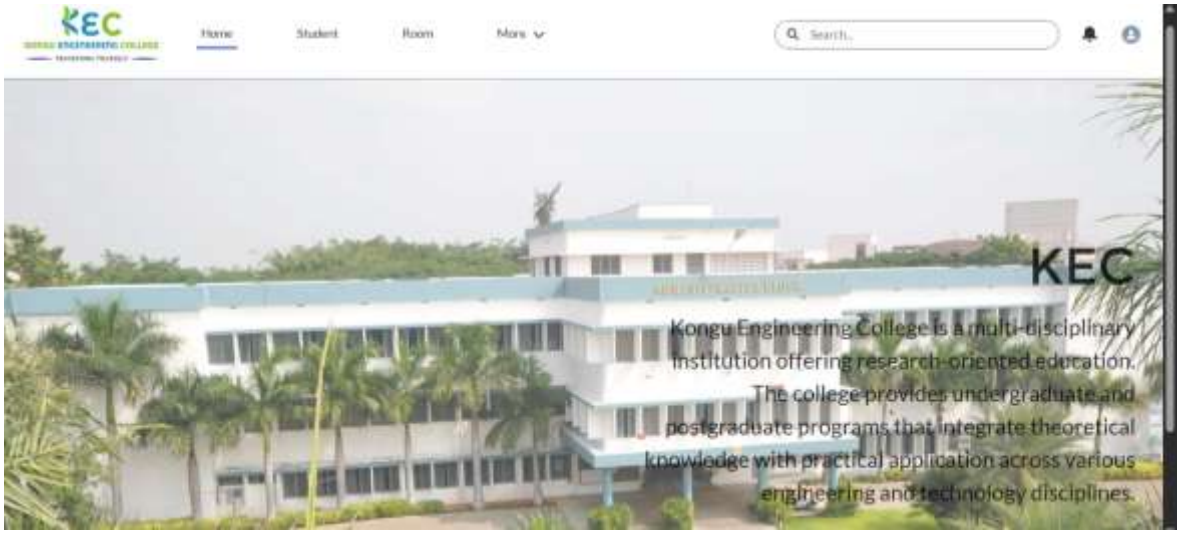

Fig 3.7 Admin Home Page

This page displays the main dashboard for administrators with an overview of all hostel operations. It shows key metrics such as total students, available rooms, pending complaints, and recent fee collections through summary cards and interactive charts. Administrators can access quick links to manage student records, room allocations, mess menus, complaints, and fee payments from this central hub.

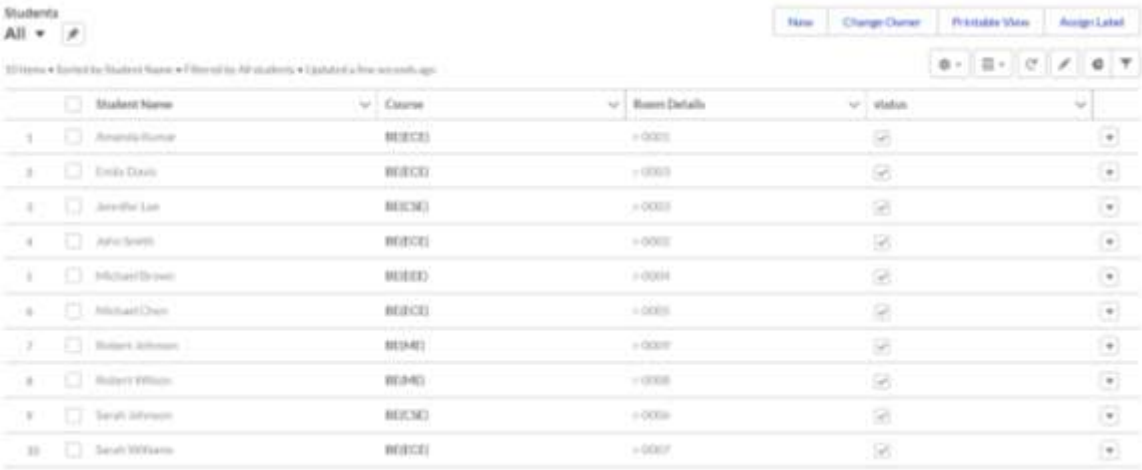

Figure 3.8 Student Details Page

This page displays the student registration form, serving as the primary interface for new students to provide the necessary personal and academic details required to complete the hostel enrollment process. The form is structured into distinct sections to capture comprehensive information efficiently: the personal details section typically includes fields such as full name, date of birth, gender, contact number, email address, and permanent address, while the academic details section captures information such as student ID, course or program of study, year or semester of enrollment, department, and institution name. Additional fields may be included for hostel-specific requirements, such as preferred room type, dietary restrictions, medical conditions, emergency contact information, and intended duration of stay.

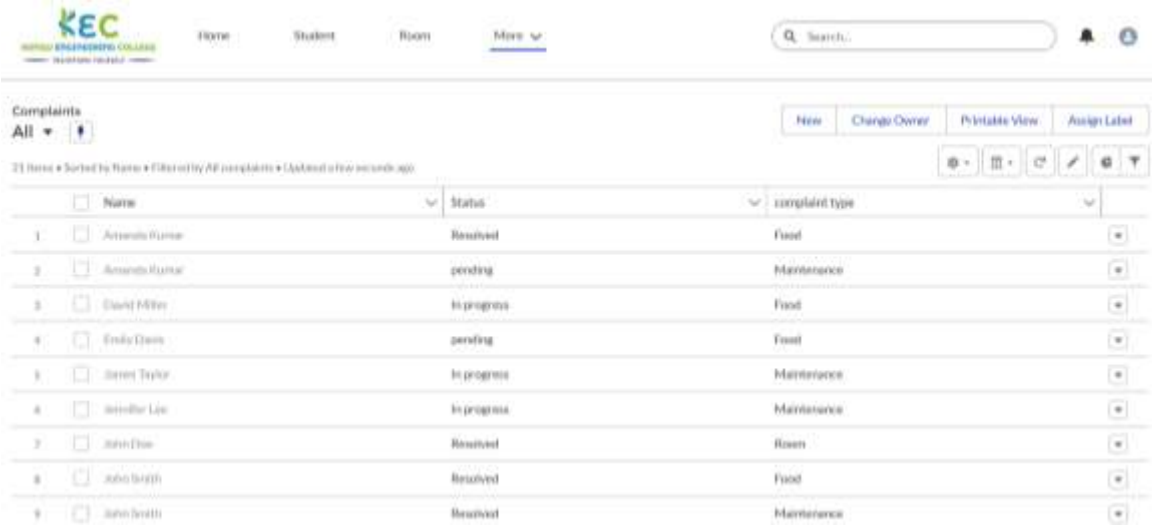

Figure 3.9 Complaint Register Page

This page displays all complaints submitted by students, providing a centralized and transparent interface for tracking the status and resolution of each issue. The complaints are typically presented in a tabular or list format, with key details such as complaint ID, submission date, category or type of issue, description, and the name of the student who filed the complaint. For each entry, the current status is clearly indicated such as "Pending," "In Review," "Assigned," "Resolved," or "Closed" allowing students to easily monitor the progress of their requests. Where applicable, resolution details are also shown, including the date of resolution, notes or comments from the administrative staff or maintenance team, and any actions taken to address the complaint.

3.6 ER DIAGRAM

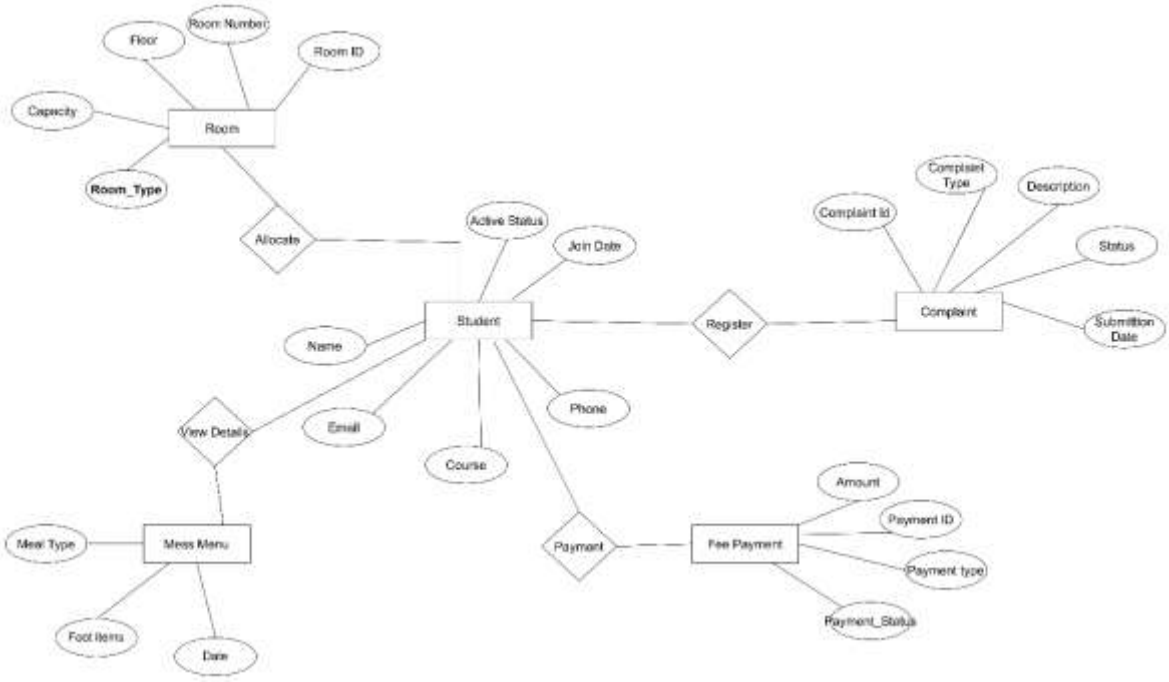

Figure 3.10 Entity Relationship Diagram

The Entity Relationship (ER) Diagram defines the logical database structure for the Hostel and Mess Management System. It organizes data by identifying key entities Student, Room, and Mess and establishing the relationships between them to ensure efficient data management and interaction. The Student entity tracks personal details like room number and complaints records application statuses. This framework allows the system to effectively manage the entire hostel lifecycle from student registration and room allocation to complaint resolution and fee tracking.

3.7 DATABASE DESIGN

The database design of the Hostel & Mess Management System is implemented using Salesforce custom objects. These objects store structured data related to student information, room details, mess menus, complaints, and fee payments. The database design ensures efficient storage, retrieval, and management of hostel-related information.The main objects used in the system include Student_Details__c, Room_Details__c, Mess_Details__c, Complaint_Details__c, and Fee_Payment__c. Complaints are linked to the students who submitted them, and fee payments are linked to the respective students. The database design improves data integrity and helps administrators manage hostel operations efficiently.

3.8 TABLE DESIGN

The table design describes the structure of the database tables used in the system. Each table contains fields that store specific information required for managing hostel and mess operations.

Table 3.1 Student Object (Student__c)

Field Name	**API Name**	**Data Type**
Student Name	Name	Text
Course	Course__c	Text
Join Date	Join_Date__c	Date
Gender	Gender__c	Picklist
Status	Status__c	Checkbox
Email	Email__c	Email
phone	phone__c	Phone
Room Details	Room_Details__c	Lookup(Room)

The Student Details object stores information of students residing in the hostel, including student name, unique student ID, email, phone, course, join date, and current status. A lookup field to Room Details connects students to their assigned rooms. These details are used for identification, communication, and tracking student occupancy throughout their stay.

Table 3.2 Room Details Object (Room_Details__c)

Field Name	**API Name**	**Data Type**
Room Number	Name	Auto Number
Floor	Floor__c	Number
Capacity	Capacity__c	Number
Available Beds	Available_Beds__c	Formula
Room Type	Room_type__c	Picklist

The Room Details object stores information of hostel rooms including room number, floor location, total capacity, available beds, and room type. The available beds field automatically calculates real-time occupancy by subtracting assigned students from total capacity. This helps administrators manage room inventory and allocate rooms efficiently.

Table 3.3 Mess Details Object (Mess_Details__c)

Field Name	**API Name**	**Data Type**
Menu Name	Name	Text
Meal Type	Meal_Type__c	picklist
Food Items	Food_items__c	Long Text Area
Monthly Fee	Monthly_Fee__c	Currency
Date	Date__c	Date

The Mess Details object stores daily mess menu information including menu name, meal type, food items served, monthly mess fee, and applicable date. Mess staff use this to update daily menus, and students view available food options through their portal. This object helps maintain organized records of meal services in the hostel mess.

Table 3.4 Complaint Details Object (complaint_Details__c)

Field Name	**API Name**	**Data Type**
Complaint Number	Name	Auto Number
Complaint type	Complaint_Type__c	picklist
Description	Description__c	Long Text Area
Status	Status__c	picklist
Date	Date__c	Date
Student Details	Student_details__c	Lookup(Student)

The Complaint Details object maintains records of student complaints including complaint number, type, description, status, submission date, resolution date, and a lookup to the student. This helps maintenance staff track and resolve issues systematically while providing students transparency about complaint status.

Table 3.5 Fee Payment Object (Fee_payment__c)

Field Name	**API Name**	**Data Type**
Payment Number	Name	Auto Number
Amount	Amount__c	Currency
Payment Date	Payment_date__c	Date
Payment type	Payment_type__c	picklist
Status	Status__c	picklist
Student Details	Students_details__C	Lookup(Student)

The Fee Payment object maintains records of all student fee payments including payment number, amount, date, type, status, and a lookup to the student. This helps accountants track collections, calculate pending amounts, generate receipts, and maintain complete financial history for each student.

SUMMARY

Chapter 3 describes the complete system design of the Smart Agri-Commerce and Farmer Support System. It explains the overall architecture and details the major functional modules such as Admin, Farmer (Customer), Shopkeeper, Delivery Agent, Product Management, Order Management, Payment Gateway, Weather Information, AI Chatbot, Government Schemes, Disease Insights, and Analytics. The chapter also includes system flow diagrams and Data Flow Diagrams (DFD Level 0) to illustrate how data moves between users, external entities, and the central system, helping to clearly define system boundaries and interactions. In addition, the chapter discusses the database design using MongoDB, focusing on data integration, data independence, and security. Overall, this chapter provides a strong technical foundation for system implementation, which is carried out in the next chapter.

CHAPTER 4

IMPLEMENTATION

4.1 CODE DESCRIPTION

Code description plays a vital role in explaining the functionality and intent of the source code used in the Hostel & Mess Management System is implemented on the Salesforce platform using Apex for backend logic, Lightning Web Components for frontend interface, and SOQL for data manipulation. Apex classes contain business logic for operations like calculating available beds, processing fee payments, assigning complaints, and generating reports. Each class follows single responsibility principle, with dedicated classes for student management, complaint handling, and fee processing.

Apex triggers execute automatically on record events to enforce business rules and maintain data integrity, such as updating related records and sending notifications when complaint status changes. Triggers delegate complex logic to handler classes to keep them simple and maintainable.

SOQL queries retrieve data efficiently from the database, fetching only required fields. Proper comments explain complex logic, database interactions, and validation mechanisms, making it easier for future developers to understand and extend the system functionality.

4.2 STANDARDIZATION OF THE CODING

Coding standards are strictly followed throughout the development of the Hostel & Mess Management System to maintain uniformity, readability, and scalability. Standard coding practices such as meaningful variable and function names, proper indentation, modular file structure, and consistent formatting are applied across both Apex backend and Lightning Web Components frontend code. Apex follows Salesforce best practices including bulk-safe triggers, separation of concerns using handler classes, and proper exception handling. Lightning Web Components use modern JavaScript standards with proper event handling and reactive programming patterns.

The following code snippet demonstrates a standardized Apex class structure used in the Hostel & Mess Management System:

```
public with sharing class RoomService {
public static List<Room_Details__c> getAvailableRooms() {
try {
List<Room_Details__c> rooms = [
SELECT Id, Name, Floor__c, Capacity__c,
Room_Type__c, Available_Beds__c
FROM Room_Details__c
WHERE Available_Beds__c > 0
ORDER BY Floor__c, Name
];
System.debug('Retrieved ' + rooms.size() + ' available rooms');
return rooms;
} catch (QueryException e) {
System.debug('Error fetching available rooms: ' + e.getMessage());
return new List<Room_Details__c>();
}
}
}
```

4.3 ERROR HANDLING

Error handling is a crucial part of the implementation to ensure system stability and a smooth user experience. In the Hostel & Mess Management System, errors may occur due to invalid user inputs, validation rule violations, insufficient permissions, database operation failures, or unexpected system exceptions. To handle such situations efficiently, structured error-handling mechanisms are implemented across the application.

The system extensively uses try-catch blocks to catch both expected and unexpected errors, preventing application crashes and allowing graceful error recovery. Caught exceptions are logged using System. debug() for debugging purposes, while user-friendly messages are displayed to guide users in resolving issues. Custom exception classes are defined for specific error scenarios, providing better error organization and clarity. Salesforce platform errors such as DmlException, QueryException, and LimitException are handled appropriately with fall back mechanisms.

The improved error handling code used in the project is shown below:

```
public static List<student_c> safeQuery(String query) {
    try {
        List<student_c> results = Database.query(query);
        System.debug('Query executed successfully. Returned ' + results.size() +
records');        return results;
    } catch (QueryException e) {
        System.debug('Query error: ' + e.getMessage());
        System.debug('Failed query: ' + query);
        return new List<sObject>();
         } catch (LimitException e) {
        System.debug('Governor limit exceeded: ' + e.getMessage());
        throw new HandledException('System limits exceeded. Please try again
later.');  }
```

This robust error-handling strategy ensures data integrity, system reliability, and improved user satisfaction.

SUMMARY

Chapter 4 described the implementation details of the Hostel & Mess Management System. It explained the code structure using Apex classes, triggers, Lightning Web Components, and SOQL queries. The chapter emphasized adherence to coding standards including proper naming conventions, code organization, and commenting practices to ensure maintainability and scalability. A sample code snippet demonstrated the standardized approach with proper documentation. Effective error-handling techniques using try-catch blocks, custom exceptions, and user-friendly messages were discussed, with sample code showing robust database operations. This error-handling strategy ensures data integrity, system reliability, and improved user satisfaction. The next chapter discusses Testing, which validates the correctness and performance of the implemented system.

CHAPTER 5

TESTING AND RESULTS

5.1 TESTING

Testing is a critical phase in the software development life cycle that ensures the reliability, correctness, and performance of the Hostel & Mess Management System. This chapter explains the testing methodologies adopted to validate that the developed system meets functional requirements and performs as expected under various conditions. Testing was carried out after the completion of development and was documented using appropriate test cases and outcomes.

The system underwent multiple levels of testing to identify and eliminate errors, verify module functionality, and ensure smooth interaction between different system components. Each module was tested individually and then as part of the integrated system.

5.1.1 UNIT TESTING

Unit testing formed the foundational layer of our testing strategy, focusing on verifying individual system components in isolation to ensure they function correctly before integration. During the development phase, each module underwent rigorous unit testing by the developer, following a test-driven approach where appropriate. The student management module was tested for registration, profile updates, and data validation; the room management module for allocation processes, availability checking, and capacity limits; and the mess management module for subscription functionality, meal planning, and fee calculations. Additionally, the complaint handling module was verified for registration, status tracking, and assignment logic, while the fee payment module underwent testing for calculations, payment gateway integration, and receipt generation. For each module, the testing approach verified input validation using valid, invalid, and boundary values; business logic to ensure conditional statements and calculations produced expected results; error handling for graceful failure; database operations for correct CRUD functionality; and return values for expected outputs.

Test Case ID	Test Object	Test Data	Expected Result	Actual Result	Test Result
TC_U01	Student Module	Add new Student details	Student record stored in system	Student record stored successfully	Pass
TC_U02	Student List Module	View Student List	System Displays Student Records	Student List Displayed correctly	Pass
TC_U03	Room Module	Add new room	Room record created	Room Stored Successfully	Pass
TC_U04	Complaint Module	Submit new Complaint	Complaint record created	Complaint Stored Successfully	Pass
TC_U05	Fee Payment Module	Record fee payment	Payment record created	Payment stored successfully	Pass
TC_U06	Mess Module	Update daily menu	Menu record updated	Menu Updated successfully	Pass

5.1.2 INTEGRATION TESTING

Integration testing verifies the interaction between multiple modules to ensure that data flows correctly across the system, serving as a critical phase in the software development lifecycle that bridges the gap between unit testing and system-level validation. In this project, a top-down integration approach was employed, beginning with the main system modules such as the user authentication module, dashboard, and core navigation components and progressively integrating subordinate sub-modules in a controlled, hierarchical manner.

Test Case ID	Test Object	Test Data	Expected Result	Actual Result	Test Result
TC_I01	StudentModule → Database	Add student details	Student record stored in database	Data stored successfully	Pass
TC_I02	Complaint module → Notification	Submit new Complaint	Notification sent to maintenance staff	Data stored successfully	Pass
TC_I03	Fee Module →Dashboard	Record fee payment	Dashboard updates fee collection report	Email notification sent	Pass
TC_I04	RoomModule → Database	Add room details	Room record stored	Dashboard updated correctly	Pass
TC_I05	Student Module →Room	Assign student to room	Student linked with room	Data stored successfully	Pass
TC_I06	Complaint Module → Student portal	Update complaint status	Student portal shows updated status	Link created successfully	Pass

5.1.3 VALIDATION TESTING

Validation testing ensures that the Hostel & Mess Management System meets user expectations and fulfills business requirements by confirming that the system performs real-world tasks effectively and aligns with the needs of students, administrators, and staff. This testing phase goes beyond verifying individual functions to evaluate whether the complete system behaves as intended in practical scenarios, such as students successfully registering for mess services, administrators efficiently allocating rooms, and staff properly managing complaints. Through validation testing, confirmed that all user interactions produce meaningful results, business rules are correctly applied during operations like fee calculations and room assignments.

Test Case ID	Test Object	Test Data	Expected Result	Actual Result	Test Result
TC_V01	Student Registration From	Empty required fields	System shows validation error	Error message displayed	Pass
TC_V02	Room Assignment	Assign to full room	System prevents assignment	“Room full” error displayed	Pass
TC_V03	Fee payment	Negative amount	System shows validation error	Error message displayed	Pass
TC_V04	Fee payment	Future payment date	System prevents future date	Validation error shown	Pass
TC_V05	Complaint submission	Empty description	System shows validation error	Duplicate error shown	Pass

SUMMARY

Chapter 5 presents the testing and results of the Hostel & Mess Management System. Unit, integration, and validation testing were carried out to verify individual module functionality, ensure proper interaction between system components, and confirm compliance with user requirements. All major modules, including student management, room management, mess management, complaint handling, fee payment, authentication, and reporting, were tested successfully, demonstrating that the system is reliable, efficient, and ready for deployment.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 CONCLUSION

The Hostel & Mess Management System, developed on the Salesforce platform using Lightning Web Components and Apex, successfully digitizes and streamlines hostel operations in educational institutions. The system integrates student management, room allocation, mess operations, complaint handling, and fee tracking into a single unified platform, addressing key challenges such as manual record keeping, delayed complaint resolution, and difficulty in tracking room availability. Administrators can efficiently manage all hostel operations through a centralized dashboard, while students benefit from easy access to room details, mess menus, complaint submission, and fee status through their dedicated portal.

Comprehensive testing confirms that all modules function correctly and interact seamlessly. The system reduces manual workload, improves data accuracy, enhances communication, and provides real-time visibility into hostel operations, demonstrating how Salesforce's low-code platform can effectively modernize institutional facilities management.

6.2 FUTURE ENHANCEMENTS

Future enhancements to the Hostel & Mess Management System can further strengthen its impact and usability. A dedicated mobile application using Salesforce Mobile SDK would allow students and staff to access the system on the go with push notifications. Automated email and SMS reminders for pending fees would improve collection rates. Smart room allocation algorithms can optimize assignments based on student preferences. Inventory management for mess supplies would track stock levels and generate purchase orders. Image uploads in complaints would help maintenance staff better understand issues. Multi-campus support would allow management of multiple hostels from a single interface. These enhancements would make the system more intelligent, efficient, and student-centric.

APPENDICES

A. SAMPLE CODING

Backend File

RoomManagement.cls

```
public with sharing class RoomManagement {
@AuraEnabled(cacheable=true)
public static List<Room_Details__c getAvailableRooms() {
    try {
        return [
            SELECT Id, Name,
            Floor__c, Capacity__c,
            Room_Type__c,
            Available_Beds__c
            FROM Room_Details__c
            WHERE Available_Beds__c > 0
            ORDER BY Floor__c, Name];
    } catch (Exception e) {
        throw new AuraHandledException('Error fetching rooms:'+e.getMessage());
    }}
@AuraEnabled
public static Boolean assignStudentToRoom(Id studentId, Id roomId) {
    try {
        Room_Details__c room = [
            SELECT Available_Beds__c
            FROM Room_Details__c
            WHERE Id = :roomId
            LIMIT 1];
        if (room.Available_Beds__c <= 0) {
            throw new AuraHandledException('Room is full');
        }
```

```
        Student_Details__c student = new Student_Details__c(
            Id = studentId,
            Room_Details__c = roomId
        );
        update student;
        return true;
    } catch (Exception e) {
        throw new AuraHandledException('Assignment failed:'+e.getMessage());
    }}}
```

ComplaintHandler.cls

```
public with sharing class ComplaintHandler {
    @AuraEnabled
    public static Id submitComplaint(Id studentId, String type, String description) {
        try {
            if (String.isBlank(description)) {
                throw new AuraHandledException('Description is required');
            }
            Complaint_Details__c complaint = new Complaint_Details__c(
                Student_Details__c = studentId,
                Complaint_Type__c = type,
                Description__c = description,
                Status__c = 'New',
                Date__c = Date.today());
            insert complaint;
            return complaint.Id;
        } catch (Exception e) {
            throw new AuraHandledException('Error submitting complaint:'+e.getMessage());
        }}
    @AuraEnabled
    public static void updateComplaintStatus(Id complaintId, String status) {
        try {
            Complaint_Details__c complaint = new Complaint_Details__c(
```

```
            Id = complaintId,
            Status__c = status);
        if (status == 'Resolved') {
            complaint.Resolution_Date__c = Date.today();
        }
        update complaint;
    } catch (Exception e) {
        throw new AuraHandledException('Error updating status:'+e.getMessage());
    }
  }
}
```

Fee PaymentHandler.cls

```
 public with sharing class FeePaymentHandler {
@AuraEnabled
public static Id recordPayment(Id studentId,Decimal amount,
                String paymentType,Date paymentDate) {
    try {
        if (amount <= 0) {
            throw new AuraHandledException('Amount must be positive');
        }
        if (paymentDate > Date.today()) {
            throw new AuraHandledException('Payment date cannot be in the future');
        }
        Fee_Payment__c payment = new Fee_Payment__c(
            Student_Details__c = studentId,
            Amount__c = amount,
            Payment_Type__c = paymentType,
            Payment_Date__c = paymentDate,
            Status__c = 'Paid'
        );
        insert payment;
        return payment.Id;
    } catch (Exception e) {
```

```
        throw new AuraHandledException('Error recording payment: ' + e.getMessage());
    }
}
@AuraEnabled(cacheable=true)
public static Decimal getPendingFees(Id studentId) {
    try {
        AggregateResult result = [
            SELECT SUM(Amount__c) totalPaid
            FROM Fee_Payment__c
            WHERE Student_Details__c = :studentId
            AND Status__c = 'Paid'];
        Decimal totalPaid = (Decimal)result.get('totalPaid');
        Decimal totalFee = 10000;
        return totalFee - (totalPaid != null ? totalPaid : 0);
    } catch (Exception e) {
        throw new AuraHandledException('Error calculating pending fees');
    } }}
```

Frontend File

studentRegistrationForm.html

```
                <template>
  <lightning-card,title="Student    Registration">
<div class="slds-p-around_medium">
        <lightning-record-edit-form
            object-api-name="Student_Details__c"
            onsuccess={handleSuccess}
            onsubmit={handleSubmit}>
            <lightning-messages></lightning-
messages>
            <div class="slds-grid slds-wrap">
                <div class="slds-col slds-size_1-of-2
                        slds-p-around_xx-small">
```

```
                <lightning-input-field          field-
name="Name"></lightning-input-field>
</div><div class="slds-col slds-size_1-of-2 slds-
p-around_xx-small">
                <lightning-input-field field-
            </div>
            <div class="slds-col slds-size_1-of-2
slds-p-around_xx-small">
                <lightning-input-figld          field-
name="Phone__c"></lightning-input-field>
            </div>
            <div class="slds-col slds-size_1-of-2
slds-p-around_xx-small">
                <lightning-input-field          field-
name="Course__c"></lightning-input-field>
 </div><div class="slds-col slds-size_1-of-2 slds-
p-around_xx-small"><lightning-input-field field-
name="Join_Date__c"></lightning-input-field>
            </div><div     class="slds-col     slds-
size_1-of-2 slds-p-around_xx-small">
                <lightning-input-field          field-
name="Room_Details__c"></lightning-input-
field></div></div>
          <div class="slds-m-top_medium">
            <lightning-button
              type="submit"
              variant="brand"
              label="Register Student">
            </lightning-button>
```

```
            </div>
        </lightning-record-edit-form>
    </div>
  </lightning-card>
</template>
```

studentRegistrationForm.js

```
import { LightningElement } from 'lwc';
import{ShowToastEvent}from
'lightning/platformShowToastEvent';
export  default  class  StudentRegistrationForm
extends LightningElement {
  handleSuccess(event) {
    this.dispatchEvent(
        new ShowToastEvent({
            title: 'Success',
            message:
            'Student registered successfully',
            variant: 'success'
        }));
    constinputField
this.template.querySelectorAll('lightning-
input-field');
    if (inputFields) {
        inputFields.forEach(field => {
            field.reset();
        });  }}
  handleSubmit(event) {
```

```
        const fields = event.detail.fields;
        if (!fields.Email__c) {
            event.preventDefault();
            this.dispatchEvent(
                new ShowToastEvent({
                    title: 'Error',
                    message: 'Email is required',
                    variant: 'error'
                })  );
        }
    }
}
```

complaintSubmissionFrom.html

```
<template>
    <lightning-card title="Submit Complaint">
        <div class="slds-p-around_medium">
<lightning-record-edit-form
    object-api-name="Complaint_Details__c"
    onsuccess={handleSuccess}>
  <lightning-messages></lightning-messages>
                <div class="slds-grid slds-wrap">
                    <div class="slds-col slds-size_1-
of-1 slds-p-around_xx-small">
  <lightning-input-field
  field-name="Student_Details__c"
                      value={studentId}>
                     </lightning-input-field>
```

```
                </div><div class="slds-col slds-
size_1-of-2 slds-p-around_xx-small">
                        <lightning-input-field field-
      name="Complaint_Type__c"
      onchange={handleTypeChange}>
                    </lightning-input-field>
                  </div>
                  <div class="slds-col slds-size_1-
   of-2 slds-p-around_xx-small">
                    <lightning-input-field field-
   name="Date__c"></lightning-input-field>
                  </div>
                  <div class="slds-col slds-size_1-
   of-1 slds-p-around_xx-small">
                    <lightning-input-field field-
   name="Description__c"></lightning-input-
   field>
    </div></div><template
   if:true={showMaintenanceTips}>
                <div class="slds-m-top_medium
slds-p-around_small slds-theme_info">
                  <strong>Maintenance
Tips:</strong> Please provide details about the
issue. Our maintenance team will attend to it
within 24 hours.
                </div>
            </template>
            <div class="slds-m-top_medium">
                <lightning-button
```

```
                type="submit"
                variant="brand"
                label="Submit Complaint">
            </lightning-button>
        </div>
    </lightning-record-edit-form>
  </div>
 </lightning-card>
</template>
```

complainSubmissionFrom.js

```
import { LightningElement, api } from 'lwc';
import{ShowToastEvent}from
'lightning/platformShowToastEvent';
export default class
ComplaintSubmissionForm extends
LightningElement {
  @api studentId;
  @api recordId;
  showMaintenanceTips = false;
  handleTypeChange(event) {
    const complaintType = event.detail.value;
    this.showMaintenanceTips
(complaintType === 'Room Maintenance');
  }
  handleSuccess(event) {
    this.dispatchEvent(
      new ShowToastEvent({
        title: 'Success',
```

```
            message:
            'Complaint submitted successfully',
            variant: 'success'
        })    );
            setTimeout(() => {
            this.dispatchEvent(new
CustomEvent('complaintsubmitted'));
      }, 2000);
   }
}
```

B. SCREENSHOTS

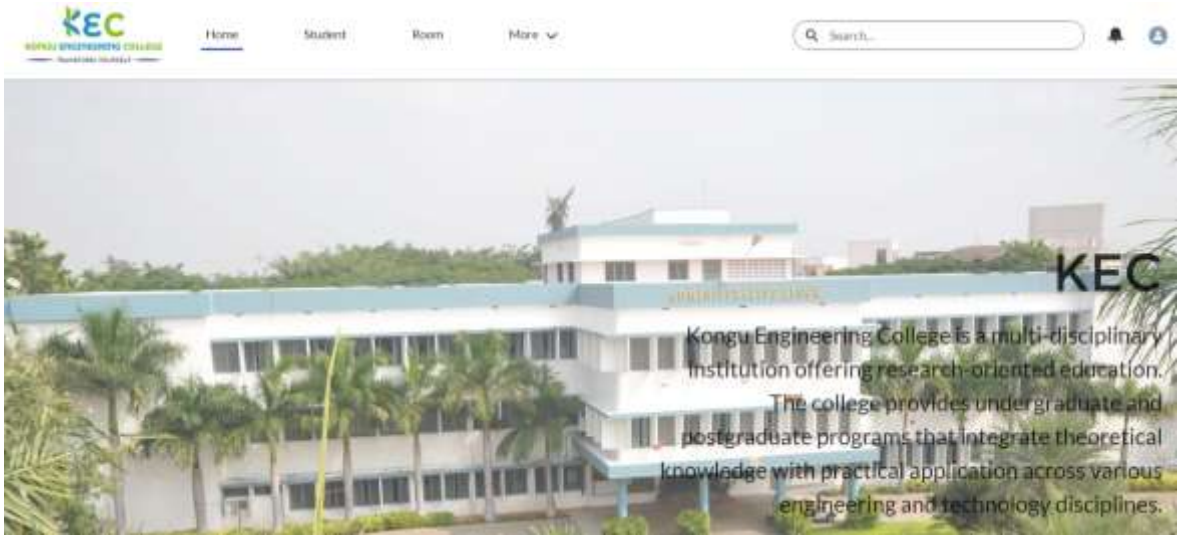

Figure B.1 Hostel and Mess Portal Home Page

This screen shows the main home page of the Hostel and Mess Management System portal. It provides an overview of hostel facilities and mess services, designed to streamline accommodation and dining management for students. The interface helps students track daily meals, manage fee payments, and stay updated with announcements. The navigation menu allows users to access modules such as Mess Menu, Room Allocation, Fee Details, and Complaints, ensuring a seamless and organized hostel living experience.

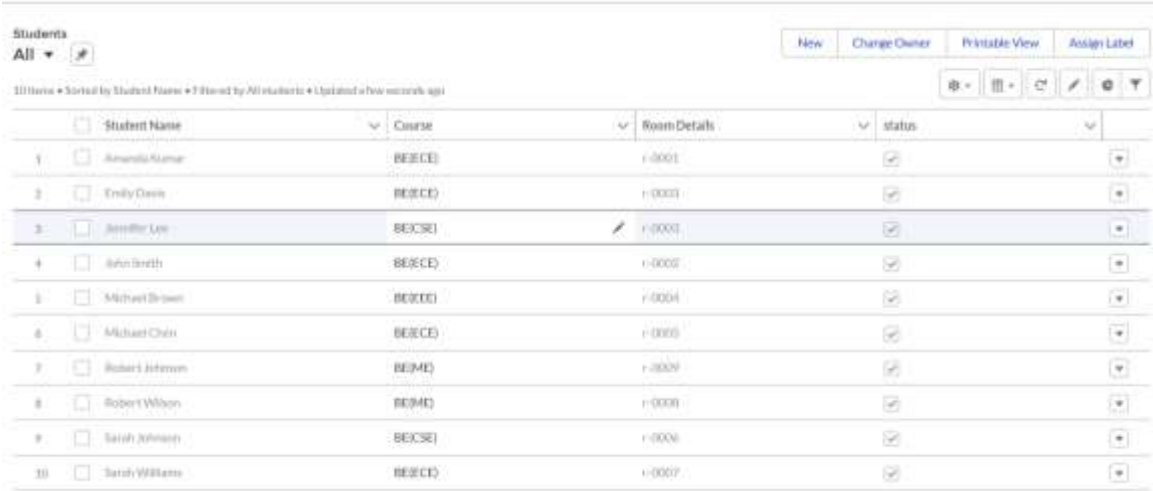

Figure B.2 Student Record Page

This screen displays the Student Record page of the Hostel and Mess Management System. It provides a detailed overview of individual student information related to hostel accommodation and mess services.

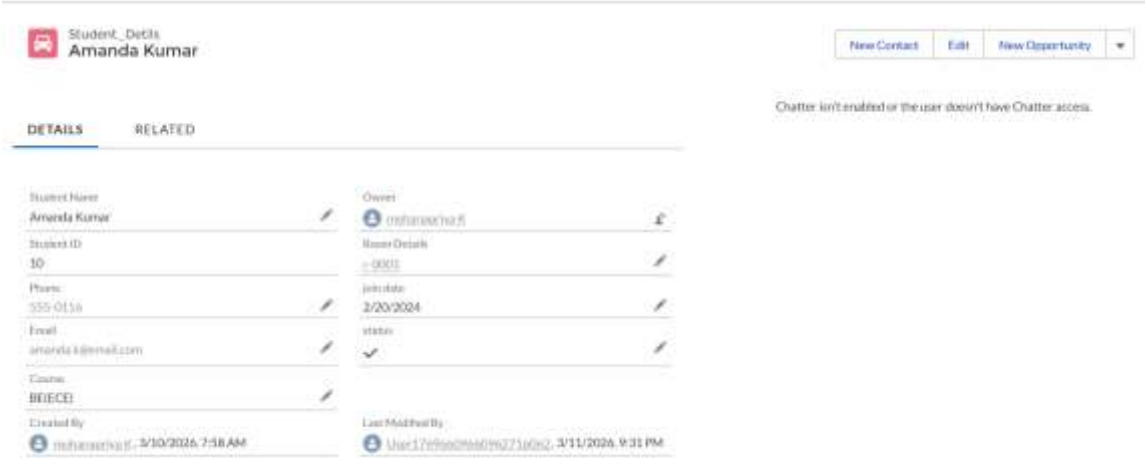

Figure B.3 student detail page

This screen displays the detailed Student Information page of the Hostel and Mess Management System. It offers a comprehensive view of a specific student's profile, consolidating all hostel and mess-related data in one place.

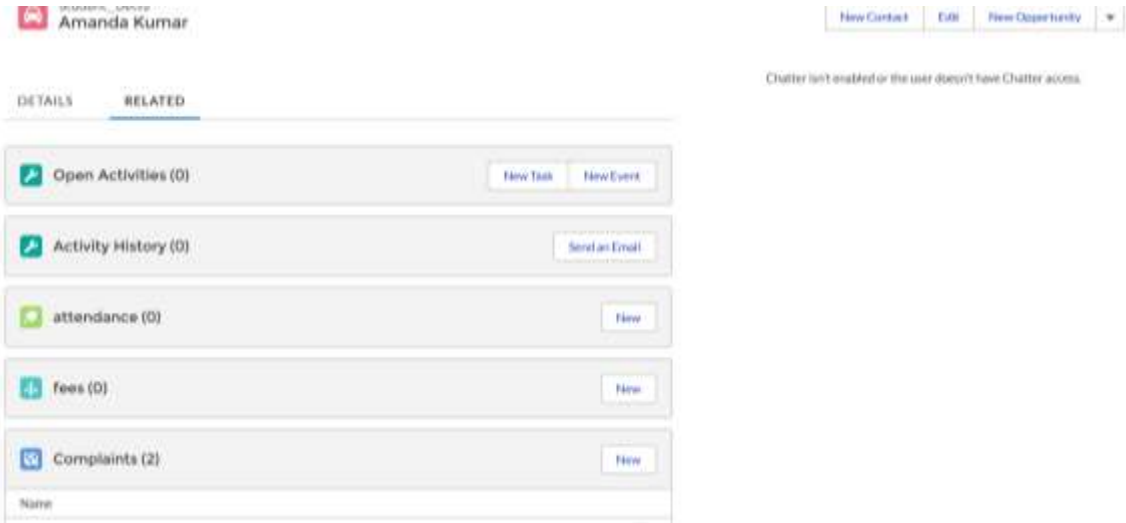

Figure B.4 Student Related page

This screen displays the Student Review and Feedback page of the Hostel and Mess Management System. It enables students to share their experiences and provide valuable input regarding hostel facilities and mess services.

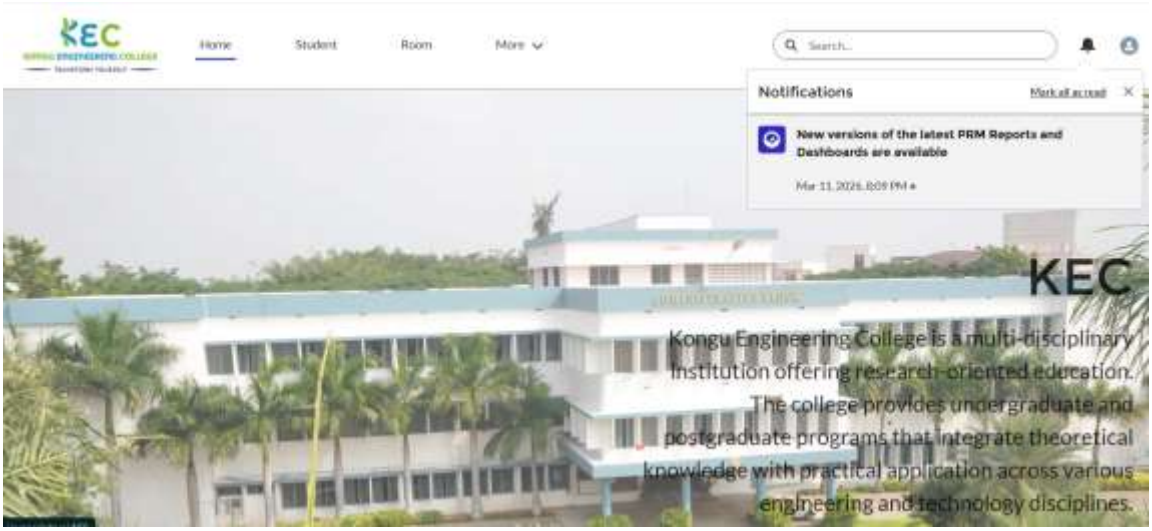

Figure B.5 View Notification

This screen displays the Notifications page of the Hostel and Mess Management System. It serves as a centralized communication hub where students and staff can view important announcements and updates from hostel administration and mess managers.

Figure B.6 Complaint Register Page

This screen displays the Complaint Register page of the Hostel and Mess Management System. It provides a structured interface for students to register grievances and for administrators to track and resolve issues efficiently.

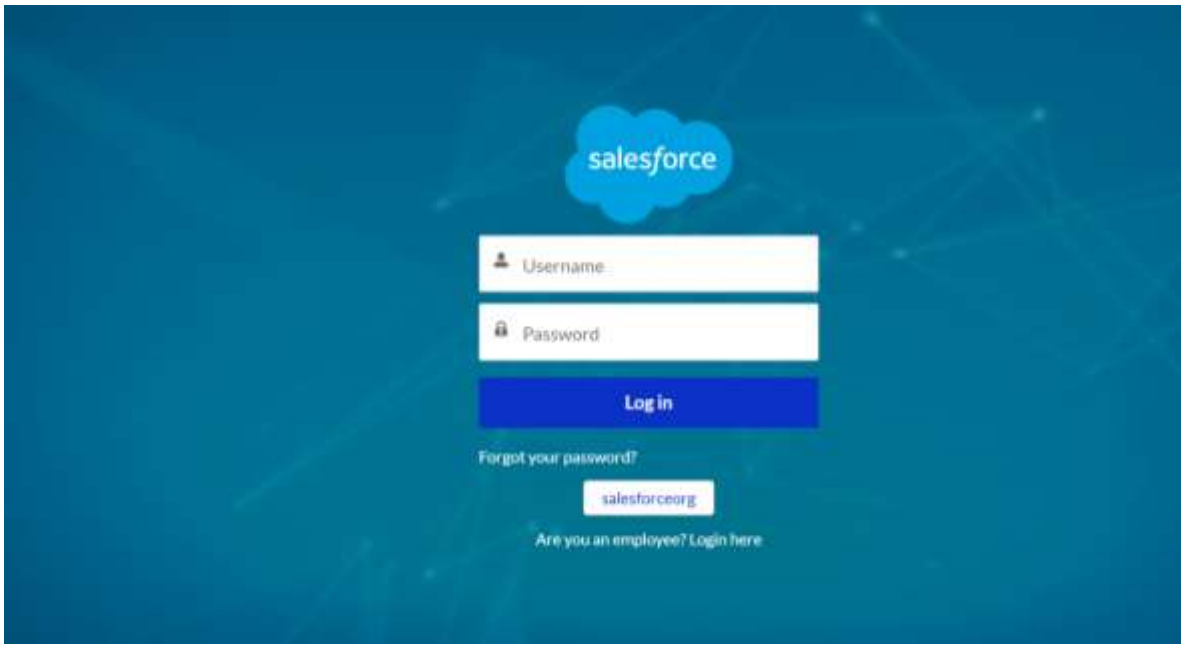

Figure B.7 Login Page

This screen displays the Login Page of the Hostel and Mess Management System. It serves as the secure entry point for students, staff, and administrators to access the portal.

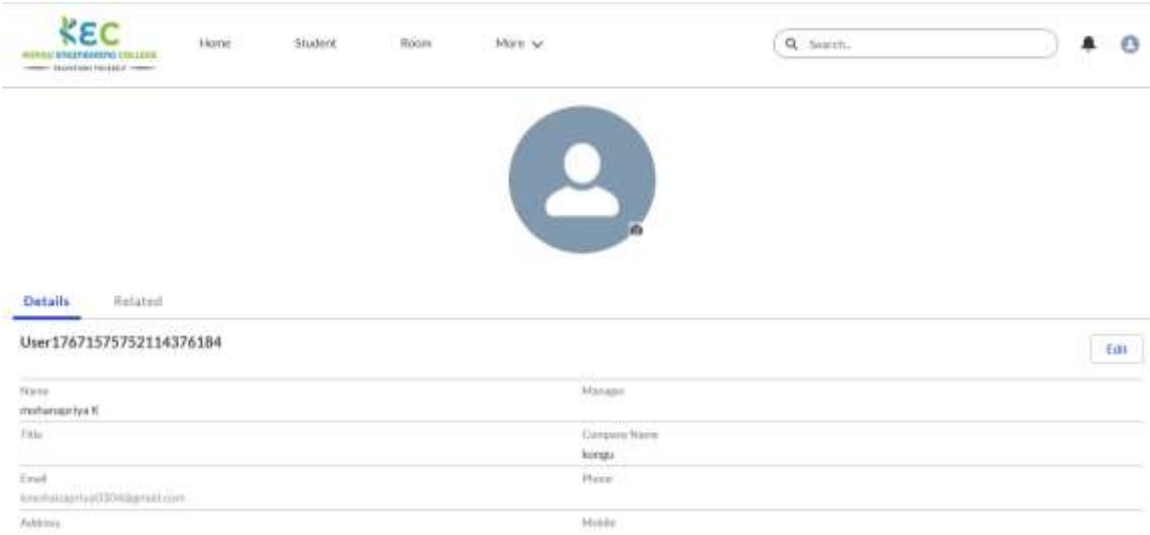

Figure B.8 Profile Page

This screen displays the User Profile Page of the Hostel and Mess Management System. It serves as a comprehensive personal dashboard where students can view and manage their account information and preferences.

REFERENCES

- Salesforce Documentation, "Lightning Web Components Developer Guide," Salesforce Inc., Available at: https://developer.salesforce.com/docs.
- Salesforce Documentation, "Apex Developer Guide," Salesforce Inc., Available at: https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode.
- Trailhead by Salesforce, "Experience Cloud Basics," Salesforce Inc., Available at: https://trailhead.salesforce.com.
- Sommarskog E., "Database Design Principles," Pearson Education, 2019.
- Pressman R. S., *Software Engineering: A Practitioner's Approach*, McGraw-Hill Education, 2014.
- Silberschatz A., Korth H. F., and Sudarshan S., *Database System Concepts*, McGraw-Hill Education, 2011.
- W3Schools, "HTML, CSS and JavaScript Tutorials," Available at: https://www.w3schools.com.
- Salesforce Trailhead, "Salesforce Platform Basics," Salesforce Inc., Available at: https://trailhead.salesforce.com/content/learn/modules/platform_basics

SUSTAINABLE DEVELOPMENT GOALS (SDGS)

SDG 3 : Good Health and Well-Being

Target: 3.4 : Reduce by one third premature mortality from non-communicable diseases through prevention and treatment and promote mental

Mapping Explanation :

The Hostel and Mess Management System plays a vital role in promoting good health and well-being among students by ensuring food safety, maintaining hygiene, supporting mental health, and enabling effective emergency response. Through its mess management module, the system oversees the planning of balanced and nutritious menus while digitally tracking inventory to prevent the use of expired or contaminated ingredients, thereby reducing the risk of foodborne illnesses and promoting long-term health through proper nutrition.

The system also enforces high standards of hygiene by scheduling and verifying routine cleaning, pest control, and waste management activities across dormitories, washrooms, and dining areas, which helps prevent the spread of communicable diseases such as typhoid, hepatitis, and other water-borne infections.